

Table of Contents

Comprehensive Question Bank: Chapter 5 — Code Testing and Debugging	2
Section A: Multiple-Choice Questions (MCQs)	2
Topic 5.1: Introduction — Software Life Cycle and Debugging Mindset	2
Topic 5.2.1: Importance of Testing — Error Types	4
Topic 5.2.2: Unit Testing — Assertions, unittest, pytest	8
Topic 5.2.3: Breakpoints and Watch Expressions	12
Topic 5.2.4: Exception Handling	14
Topic 5.3: Profiling and Optimization	17
Section B: Short Answer Questions (SQs)	20
Topic 5.1: Introduction — Software Life Cycle and Debugging Mindset	20
Topic 5.2.1: Error Types — Syntax, Runtime, Logic	21
Topic 5.2.2: Unit Testing — Assertions, unittest, pytest	22
Topic 5.2.3: Breakpoints and Watch Expressions	23
Topic 5.2.4: Exception Handling	23
Topic 5.3: Profiling and Optimization	24
Section C: Long / Extensive Questions (LQs)	24
Topic 5.1: Introduction — Software Life Cycle and Debugging Mindset	25
Topic 5.2.1: Error Types — Syntax, Runtime, Logic	25
Topic 5.2.2: Unit Testing — Assertions, unittest, pytest	26
Topic 5.2.3: Breakpoints and Watch Expressions	28
Topic 5.2.4: Exception Handling	29
Topic 5.3: Profiling and Optimization	30

Comprehensive Question Bank: Chapter 5 — Code Testing and Debugging

Grade 12 Computer Science | Complete Assessment Bank

Covers: Software Life Cycle · Error Types · Unit Testing · Breakpoints & Watches · Exception Handling · Profiling & Optimization

Taxonomy: Knowledge · Comprehension · Application · Analysis · Synthesis · Evaluation

Section A: Multiple-Choice Questions (MCQs)

Topic 5.1: Introduction — Software Life Cycle and Debugging Mindset

1. What is the correct order of stages in the Software Development Life Cycle?

A) Write → Test → Design → Deploy

B) Design → Write → Test → Deploy

C) Test → Design → Write → Deploy

D) Write → Deploy → Test → Design

1. Which famous event gave the word "bug" its meaning in computing?

A) The Ariane 5 rocket explosion in 1996

B) A physical moth trapped in the Harvard Mark II computer in 1947

C) The Therac-25 radiation machine failure in 1985

D) The Knight Capital trading disaster in 2012

1. Who is credited with finding and logging the "first actual case of bug being found"?

A) Rear Admiral Grace Hopper

B) Alan Turing

C) Bill Gates

D) Donald Knuth

1. The Ariane 5 rocket explosion in 1996 was caused by:

A) A missing semicolon in a C program

B) An unhandled exception during conversion of a 64-bit float to a 16-bit integer

C) A logic error in the fuel calculation module

D) A syntax error detected only after launch

1. The Ariane 5 disaster cost approximately how much money?

A) \$37 million

B) \$3.7 billion

C) \$370 million

D) \$37 billion

1. How many seconds after liftoff did the Ariane 5 rocket self-destruct?

A) 37 seconds

B) 3.7 seconds

C) 370 seconds

D) 73 seconds

1. The Therac-25 machine delivered radiation doses how many times higher than intended due to its software bug?

A) 10 times higher

B) 100 times higher

C) 1000 times higher

D) 50 times higher

1. The Knight Capital trading disaster (2012) resulted in losses of approximately:

A) \$44 million in 45 hours

B) \$440 million in 45 minutes

C) \$4.4 billion in 4 hours

D) \$44 billion in 45 seconds

1. Which of the following best describes the role of **testing** in software development?

A) Finding the fastest algorithm for a program

B) Writing the initial version of the code

C) Verifying that the program produces correct output for given inputs

D) Deploying the program to real users

1. Which of the following best describes the role of **debugging**?

A) Writing automated test cases

B) Measuring program execution speed

C) Finding and fixing mistakes when a program does not work correctly

D) Deploying software to production servers

2. The detective debugging process begins with which step?
- A) OBSERVE — identify the wrong output or error message**
 - B) FIX — change the code immediately
 - C) HYPOTHESIZE — guess the solution first
 - D) DEPLOY — release the code to users
3. In the Software Development Life Cycle, which stages often take MORE time than writing the code itself?
- A) Design and Writing
 - B) Testing and Deployment**
 - C) Design and Testing
 - D) Writing and Deployment
4. According to the detective debugging process, what should you do AFTER forming a hypothesis?
- A) Fix the code immediately
 - B) Restart the entire program
 - C) Test — set a breakpoint or add a test and run the code**
 - D) Redeploy the software
5. Which of the following is the LAST step in the detective debugging process?
- A) Fix — change the code
 - B) Analyze — check if hypothesis matches
 - C) Verify — run the full test suite**
 - D) Observe — identify the error
6. A software engineer spends more of their daily time doing which of the following?
- A) Writing original code from scratch
 - B) Designing database schemas
 - C) Debugging and reading error stack traces**
 - D) Deploying applications to servers
-

Topic 5.2.1: Importance of Testing — Error Types

1. Which type of programming error occurs when code breaks the grammar rules of Python?
- A) Syntax error**
 - B) Logic error
 - C) Runtime error
 - D) Semantic error
2. A syntax error is detected:

- A) While the program is running
 - B) After the program produces wrong output
 - C) Before the program runs, during parsing**
 - D) Only during deployment
3. Which symbol does Python use in error messages to point directly to the location of a syntax error?
- A) * (asterisk)
 - B) ! (exclamation mark)
 - C) ^ (caret)**
 - D) # (hash)
4. A runtime error is also known as:
- A) A logic error
 - B) A syntax error
 - C) An exception**
 - D) A compile-time error
5. Which type of error occurs during program execution and causes the program to stop unexpectedly?
- A) Syntax error
 - B) Logic error
 - C) Runtime error**
 - D) Import error
6. A student writes `score > 50` instead of `score >= 50` in a grading function. The program runs without crashing but gives wrong results for a score of exactly 50. This is:
- A) A syntax error
 - B) A runtime error
 - C) A logic error**
 - D) A ZeroDivisionError
7. Which type of error is generally the most difficult to detect because Python gives no error message?
- A) Syntax error
 - B) Runtime error
 - C) Logic error**
 - D) ImportError
8. What is a **stack trace**?

- A) A visual diagram of the program's flowchart
 - B) A list of all variables in the program
 - C) A report showing the sequence of function calls that led to an error**
 - D) A tool that measures execution speed
9. When reading a Python stack trace, you should read it:
- A) From bottom to top — the bottom line shows the actual error type**
 - B) From top to bottom — the top line shows the actual error type
 - C) Only the middle section is relevant
 - D) Stack traces cannot be read in any particular order
10. Which exception is raised when you try to divide a number by zero in Python?
- A) ZeroDivisionError**
 - B) ValueError
 - C) ArithmeticError
 - D) MathError
11. Which exception is raised when you try to convert `"hello"` to an integer using `int("hello")`?
- A) TypeError
 - B) ValueError**
 - C) NameError
 - D) AttributeError
12. Which exception is raised when you try to access a list index that does not exist?
- A) KeyError
 - B) ValueError
 - C) IndexError**
 - D) RangeError
13. Which exception is raised when you access a dictionary key that does not exist?
- A) IndexError
 - B) ValueError
 - C) KeyError**
 - D) AttributeError
14. Which exception is raised when a variable is used before it is defined?
- A) TypeError
 - B) ValueError
 - C) NameError**
 - D) AttributeError
15. Which exception is raised when you try to open a file that does not exist?

A) FileNotFoundError

B) IOError

C) PathError

D) MissingFileError

16. Which exception is raised when you try to perform an operation on incompatible types (e.g., adding a string to an integer)?

A) ValueError

B) TypeError

C) AttributeError

D) OperationError

17. In the restaurant analogy for error types, a "Logic Error" is compared to:

A) A misspelling on the menu that the printer rejects

B) Running out of ingredients mid-cooking

C) Swapping salt and sugar — the dish is cooked but tastes wrong

D) Forgetting to turn on the oven

18. In the restaurant analogy, a "Syntax Error" is compared to:

A) A misspelling on the menu that the printer system rejects

B) Running out of ingredients mid-cooking

C) Swapping salt and sugar in the dish

D) Serving the wrong table

19. In the restaurant analogy, a "Runtime Error" is compared to:

A) A misspelling on the menu

B) Swapping salt and sugar

C) Running out of flour halfway through baking a cake

D) Forgetting to write the menu

20. Which tool is BEST suited for finding logic errors in Python?

A) Reading the error message carefully

B) Checking syntax highlighting in the IDE

C) Unit tests and breakpoint debugging

D) Running `cProfile`

21. The best tool for finding runtime errors is:

A) Unit testing only

B) Profiling with cProfile

C) Reading the stack trace and using exception handling

D) Running the program faster

22. Testing that prevents new code from breaking previously working features is called:

- A) Unit testing
- B) Regression testing**
- C) Edge case testing
- D) Integration testing

23. An off-by-one error is most commonly classified as which type of error?

- A) Syntax error
 - B) Runtime error
 - C) Logic error**
 - D) Import error
-

Topic 5.2.2: Unit Testing — Assertions, unittest, pytest

1. What is the primary purpose of **unit testing**?

- A) To test the entire application as one system
- B) To measure execution speed of functions
- C) To test individual functions in isolation and verify their outputs**
- D) To deploy software to production

2. What is an **assertion** in the context of testing?

- A) A function that measures execution time
- B) A class that groups related tests
- C) A statement that checks whether a condition is true and raises an error if not**
- D) A comment explaining what a function should do

3. What does the `assert` keyword do when the condition is FALSE?

- A) Prints a warning message and continues
- B) Returns `None`
- C) Raises an AssertionError and stops execution**
- D) Skips to the next line silently

4. What does the `assert` keyword do when the condition is TRUE?

- A) Raises an AssertionError
- B) Prints "Test Passed"
- C) Passes silently — nothing happens**
- D) Returns True to the caller

5. The `unittest` module in Python is:

- A) An external library that must be installed with pip
B) A built-in Python module that provides a framework for test cases
C) A paid debugging tool for professional developers
D) A module specifically for testing database connections
6. In `unittest`, test methods must begin with which prefix for automatic discovery?
A) `check_`
B) `run_`
C) `test_`
D) `assert_`
7. In `unittest`, what class must your test class inherit from?
A) `unittest.Test`
B) `unittest.Runner`
C) `unittest.TestCase`
D) `unittest.Module`
8. In `unittest`, which method checks that two values are equal?
A) `assertEqual(a, b)`
B) `assertSame(a, b)`
C) `assertMatch(a, b)`
D) `assertCompare(a, b)`
9. In `unittest`, which method checks that a specific exception is raised?
A) `assertException(Error)`
B) `assertFails(Error)`
C) `assertRaises(Error)`
D) `assertError(Error)`
10. In `unittest`, which method checks that two floating-point values are approximately equal?
A) `assertEqual(a, b, decimals=3)`
B) `assertFloat(a, b)`
C) `assertAlmostEqual(a, b, places=3)`
D) `assertClose(a, b)`
11. In `unittest`, what does a single `.` (dot) in the output represent?
A) An error in the test
B) A skipped test
C) One test that passed
D) One test file loaded
12. When `unittest` output shows `FAILED (failures=1)`, this means:

- A) The entire program has a syntax error
 - B) One test method's assertion did not hold — a bug was caught**
 - C) The test file could not be found
 - D) Python could not import the module being tested
13. `pytest` is different from `unittest` in that:
- A) pytest requires test classes; unittest does not
 - B) pytest cannot test for raised exceptions
 - C) pytest uses standalone functions starting with `test_`; no class required**
 - D) pytest only works with Python 3.10 and above
14. How is `pytest` installed?
- A) It is built into Python — no installation needed
 - B) It comes pre-installed with VS Code
 - C) Using the command `pip install pytest`**
 - D) It is downloaded from the Python official website manually
15. Which command runs pytest in verbose mode, showing each test name individually?
- A) `pytest --verbose`
 - B) `pytest -detail`
 - C) `pytest test_file.py -v`**
 - D) `pytest -show`
16. In `pytest`, how do you check that a function raises a specific exception?
- A) `assert raises(ZeroDivisionError)`
 - B) `expect(ZeroDivisionError): function()`
 - C) `with pytest.raises(ZeroDivisionError): function()`**
 - D) `assertRaises(ZeroDivisionError, function)`
17. What is an **edge case** in software testing?
- A) An error that occurs at the edge of the screen in GUI programs
 - B) An input value at the extreme boundary of what is valid**
 - C) A test that runs only on the first and last line of a function
 - D) A special type of runtime error
18. Which of the following is NOT a typical edge case to test for a function accepting numbers?
- A) Zero as input
 - B) Negative values
 - C) Very large values
 - D) The variable name used in the function**
19. Which of the following best describes **regression testing**?

- A) Testing only the newest features added to a program
 - B) Re-running all tests after a change to ensure existing features still work**
 - C) Testing that a program runs faster than before
 - D) Testing only the parts of code that contain loops
20. A test case that verifies `calculate_discount(100, 100) == 0.0` is testing:
- A) A normal case
 - B) A boundary/edge case (100% discount)**
 - C) An invalid input case
 - D) A regression case
21. A test case that verifies `calculate_discount(-50, 10)` raises a `ValueError` is testing:
- A) A normal case
 - B) A boundary case
 - C) An invalid input case**
 - D) A performance case
22. What is a **test suite**?
- A) A single test function in pytest
 - B) A collection of multiple test cases grouped together**
 - C) The output report generated after running tests
 - D) The IDE panel that displays test results
23. Which `unittest` method verifies that a value IS `None`?
- A) `assertEmpty(x)`
 - B) `assertZero(x)`
 - C) `assertIsNone(x)`**
 - D) `assertBlank(x)`
24. Which `unittest` method verifies that a condition is `False`?
- A) `assertNot(x)`
 - B) `assertFail(x)`
 - C) `assertFalse(x)`**
 - D) `assertNegative(x)`
25. In the car factory analogy for unit testing, individual "parts" being tested represent:
- A) Complete programs
 - B) Individual functions tested in isolation**
 - C) Entire databases
 - D) User interface screens
26. Which of the following is a key advantage of automated unit tests over manual testing?

- A) Automated tests are always faster to write than manual tests
 - B) Automated tests do not require any programming knowledge
 - C) Automated tests can be re-run instantly every time the code changes**
 - D) Automated tests can replace all forms of human review
-

Topic 5.2.3: Breakpoints and Watch Expressions

1. What is a **breakpoint** in an IDE debugger?
 - A) A section of code that always causes a runtime error
 - B) A marker that pauses program execution at a specific line**
 - C) A variable whose value is being monitored continuously
 - D) A function that slows down execution for testing
2. In VS Code, how do you set a breakpoint on a line of code?
 - A) Right-click the line and select "Run to here"
 - B) Press F10 while the cursor is on the line
 - C) Click in the left margin next to the line number**
 - D) Type `breakpoint()` at the end of the line
3. In PyCharm, how do you set a breakpoint?
 - A) Press Ctrl+B on the target line
 - B) Click the grey area to the right of the line number**
 - C) Go to Run → Add Breakpoint from the menu
 - D) Drag the line number to the Debug panel
4. In PyCharm, which keyboard shortcut starts the debugger?
 - A) F5
 - B) Ctrl+D
 - C) Shift+F9**
 - D) Alt+F10
5. When a program pauses at a breakpoint, execution pauses:
 - A) BEFORE executing the line where the breakpoint is placed**
 - B) AFTER executing the line where the breakpoint is placed
 - C) At the next function call after the breakpoint
 - D) At the end of the current function
6. What is a **watch expression**?

- A) A timer that measures how long a line of code takes to execute
 - B) A variable name or expression that the debugger tracks and updates continuously**
 - C) A special type of breakpoint that only triggers on certain conditions
 - D) A tool that captures all output printed to the console
7. Watch expressions are updated:
- A) Only when you manually refresh the watch panel
 - B) Once per second during execution
 - C) Automatically every time you step to the next line**
 - D) Only at the end of each function
8. The **Step Over** debugger action (F10) does which of the following?
- A) Executes the current line and pauses at the next line, without entering called functions**
 - B) Enters any function called on the current line
 - C) Finishes the current function and returns to the caller
 - D) Continues execution until the next breakpoint
9. The **Step Into** debugger action (F11) does which of the following?
- A) Executes the current line without entering called functions
 - B) Enters the function being called on the current line**
 - C) Finishes the current function and returns to the caller
 - D) Runs all remaining code without pausing
10. The **Step Out** debugger action (Shift+F11) does which of the following?
- A) Executes the current line and pauses at the next line
 - B) Enters the function being called on the current line
 - C) Finishes executing the current function and returns to the line that called it**
 - D) Exits the debugger completely
11. The **Resume/Continue** debugger action (F5) does which of the following?
- A) Exits the program immediately
 - B) Restarts the program from the beginning
 - C) Continues running until the next breakpoint is hit or the program ends**
 - D) Steps over the next 10 lines automatically
12. What is an **off-by-one error**?
- A) A runtime error caused by dividing by one
 - B) A logic error where a loop starts or ends one position off from the correct value**
 - C) A syntax error caused by missing one quotation mark
 - D) An error where a function is called one extra time

13. Which advantage does a debugger with breakpoints have over using `print()` statements for debugging?
- A) `print()` statements permanently alter the code and cannot be removed
 - B) Debuggers run the program faster than using `print()` statements
 - C) Debuggers show ALL variable values simultaneously without modifying the code**
 - D) `print()` statements cannot show integer values
14. In VS Code, which keyboard shortcut starts the debugger?
- A) F5**
 - B) F10
 - C) F11
 - D) Ctrl+D
15. When would you choose **Step Into** over **Step Over**?
- A) When you want to skip an entire loop
 - B) When you want to see the program finish faster
 - C) When you want to examine what happens inside a function being called**
 - D) When you want to stop the debugger immediately
16. The Variables panel in the debugger during a paused session shows:
- A) Only the variable you are currently watching
 - B) Only variables in the main function
 - C) All variables currently in scope at the paused line**
 - D) Only variables that have changed since the last step
-

Topic 5.2.4: Exception Handling

1. What is the purpose of exception handling in Python?
- A) To make programs run faster by skipping error-prone code
 - B) To find and automatically fix bugs in the code
 - C) To anticipate runtime errors and respond to them gracefully instead of crashing**
 - D) To prevent logic errors from occurring
2. Which Python keyword begins an exception handling block?
- A) try**
 - B) catch
 - C) handle
 - D) begin
3. Which Python keyword is used to catch a specific exception type?

- A) `catch`
 - B) `handle`
 - C) `except`**
 - D) `error`
4. The `else` block in a `try-except-else-finally` structure runs:
- A) Always, regardless of exceptions
 - B) Only when an exception is raised
 - C) Only when NO exception is raised in the `try` block**
 - D) Only when the `finally` block has completed
5. The `finally` block in a `try-except-finally` structure runs:
- A) Only when an exception is raised
 - B) Only when no exception is raised
 - C) Always, regardless of whether an exception occurred**
 - D) Only when the program is about to exit
6. In which scenario is the `finally` block most useful?
- A) When you want to repeat the try block if it fails
 - B) When you want to catch multiple exception types
 - C) When you need to close a file or database connection no matter what happens**
 - D) When you want to restart the program after an error
7. Which statement correctly raises a `ValueError` in Python?
- A) `throw ValueError("Invalid")`
 - B) `error ValueError("Invalid")`
 - C) `raise ValueError("Invalid")`**
 - D) `except ValueError("Invalid")`
8. What is a **custom exception**?
- A) A built-in Python exception with a modified error message
 - B) A user-defined exception class that inherits from Python's `Exception` class**
 - C) An exception that only triggers on specific operating systems
 - D) An exception raised only by the `raise` keyword without a class name
9. To define a custom exception called `InvalidAgeError`, which class must it inherit from?
- A) `RuntimeError`
 - B) `BaseClass`
 - C) `Exception`**
 - D) `Error`
10. When you write `except ValueError as e:`, the variable `e` contains:

- A) The line number where the error occurred
 - B) The name of the function that raised the error
 - C) The exception object, which includes the error message**
 - D) The type name as a string
11. Which of the following correctly catches both `ValueError` and `TypeError` in one `except` block?
- A) `except ValueError, TypeError:`
 - B) `except (ValueError, TypeError):`**
 - C) `except ValueError or TypeError:`
 - D) `except ValueError | TypeError:`
12. What happens if a `try` block raises an exception and there is NO matching `except` block?
- A) The exception is silently ignored
 - B) The program restarts from the beginning
 - C) The exception propagates upward and may crash the program**
 - D) Python automatically handles it and continues
13. If the Ariane 5 rocket's software had used a `try-except` block for the integer conversion, the disaster:
- A) Would have still occurred because try-except blocks cannot handle hardware errors
 - B) Could have been prevented by gracefully handling the overflow instead of crashing**
 - C) Would have been delayed but not prevented
 - D) Would have had no effect since the backup system also failed
14. The `try` block should contain:
- A) Code that might raise an exception**
 - B) Only the error message to display
 - C) The cleanup code that must always run
 - D) Code that has already been tested and confirmed to work
15. What is the difference between `except Exception` and `except ValueError`?
- A) They are identical — both catch all exceptions
 - B) `except Exception` only catches syntax errors
 - C) `except Exception` catches almost all exception types; `except ValueError` catches only `ValueError`**
 - D) `except ValueError` is the preferred way to catch all exceptions
16. Which block of the exception handling structure provides a success confirmation message?

- A) `try` block
 - B) `except` block
 - C) **`else` block**
 - D) `finally` block
17. In a database application, where should the `connection.close()` call be placed?
- A) In the `try` block only
 - B) In the `except` block only
 - C) In the `else` block only
 - D) **In the `finally` block, so it always runs**
18. What does `raise` do when used inside an `except` block with no argument?
- A) Creates a new generic exception
 - B) Cancels the current exception
 - C) **Re-raises the current exception to be handled by a caller**
 - D) Converts the exception to a warning
-

Topic 5.3: Profiling and Optimization

1. What is **profiling** in the context of Python programming?
- A) Creating a user profile in the application
 - B) Writing comments describing each function's purpose
 - C) **Measuring a program's execution time and resource usage per function to find bottlenecks**
 - D) Testing all functions with automated assertions
2. What is a **performance bottleneck**?
- A) **A part of the code that is disproportionately slower than the rest, limiting overall speed**
 - B) A syntax error that causes the program to run slowly
 - C) A function that uses too many local variables
 - D) Any function that is called more than 100 times
3. Which Python module is used for detailed, function-level profiling?
- A) `timeit`
 - B) `profile_tool`
 - C) **`cProfile`**
 - D) `speedtest`
4. Which Python module is used to compare the execution time of two specific code snippets?

- A) `timeit`
 - B) `cProfile`
 - C) `time_compare`
 - D) `benchmark`
5. In `cProfile` output, the column `totttime` represents:
- A) The total time the program has been running
 - B) The total time spent inside that function, NOT counting time in sub-functions it called**
 - C) The cumulative time including all functions called by that function
 - D) The average time per call across all calls
6. In `cProfile` output, the column `cumtime` represents:
- A) The total time inside the function only
 - B) The average time per call
 - C) The total time INCLUDING all functions called by this function**
 - D) The number of times the function was called
7. In `cProfile` output, the column `ncalls` represents:
- A) The total execution time
 - B) The cumulative time
 - C) How many times that function was called**
 - D) The number of exceptions raised inside the function
8. Which function was identified as the bottleneck in the `slow_sum` vs `fast_sum` example?
- A) `slow_sum()` — because it uses a manual loop, taking 80% of total time**
 - B) `fast_sum()` — because it uses an external function
 - C) `main()` — because it calls both functions
 - D) `process_data()` — because it is called from `main()`
9. Why is `fast_sum()` using Python's built-in `sum()` faster than `slow_sum()` using a loop?
- A) `sum()` uses parallel processing automatically
 - B) `sum()` is implemented in C — a compiled, low-level language — making it much faster than Python loops**
 - C) `sum()` skips checking each element
 - D) `sum()` only works with integer lists, making it specialized
10. What is the correct professional sequence when optimizing a slow program?
- A) Optimize the smallest function first, then work upward
 - B) Optimize all functions simultaneously
 - C) Profile first → identify bottleneck → refactor → profile again → run tests**
 - D) Rewrite the program completely before profiling

11. You should NEVER optimize code before:
- A) Writing documentation for the functions
 - B) Adding more comments to the code
 - C) Running a profiler to measure where the time is actually being spent**
 - D) Deploying the code to a test server
12. A nested loop with `range(10000)` for both inner and outer loops performs how many total operations?
- A) 10,000
 - B) 20,000
 - C) 100,000,000 (10,000 × 10,000)**
 - D) 1,000,000
13. What is the time complexity of checking `if item in my_list` for a list of `n` items?
- A) $O(n)$ — the entire list may need to be scanned**
 - B) $O(1)$ — Python lists use hash tables
 - C) $O(\log n)$ — lists are always sorted
 - D) $O(n^2)$ — requires two passes
14. What is the time complexity of checking `if item in my_set` for a set of `n` items?
- A) $O(n)$ — the entire set is scanned
 - B) $O(\log n)$ — sets use binary search
 - C) $O(1)$ — sets use hash tables for constant-time lookup**
 - D) $O(n^2)$ — requires all pairs to be checked
15. What is **refactoring**?
- A) Completely rewriting a program in a different programming language
 - B) Adding more features to a program to make it faster
 - C) Rewriting code to be faster or more efficient without changing what it does**
 - D) Removing all comments from a program to reduce file size
16. What is **memory profiling**?
- A) Testing if a program remembers previous user inputs
 - B) Measuring how much RAM a program uses per object or data structure**
 - C) Counting how many variables a function uses
 - D) Checking if a program's output matches expected results
17. Which uses significantly less memory: a Python list or a Python generator?
- A) A list — because it is faster to access
 - B) A generator — because it creates values one at a time rather than storing all at once**
 - C) They use the same amount of memory
 - D) A list — because Python optimizes list storage

18. In the race car pit stop analogy for profiling, what does measuring each part of the race represent?
- A) Running unit tests on each function
 - B) Profiling each function to see where time is being spent**
 - C) Setting breakpoints at each function call
 - D) Writing exception handlers for each stage
19. A binary search on 1,000,000 elements is approximately how many times faster than a linear search?
- A) 10 times faster
 - B) 1,000 times faster
 - C) Approximately 28,000 times faster**
 - D) 2 times faster
20. The `time` module's `time.time()` function is used to:
- A) Record the current timestamp so the difference before and after code gives elapsed time**
 - B) Pause program execution for a specified number of seconds
 - C) Format dates and times for display
 - D) Profile every function in a program automatically
21. After refactoring code to make it faster, you should ALWAYS:
- A) Delete the old version of the function
 - B) Add a comment explaining why the new code is faster
 - C) Run all unit tests to verify the output is still correct**
 - D) Run cProfile again before testing correctness
22. The `timeit` module runs a code snippet multiple times (e.g., `number=10`) in order to:
- A) Check that the function produces the correct output 10 times
 - B) Reduce measurement noise and get a more accurate average execution time**
 - C) Guarantee that no exceptions are raised
 - D) Apply the optimization automatically after 10 runs
-

Section B: Short Answer Questions (SQs)

Topic 5.1: Introduction — Software Life Cycle and Debugging Mindset

1. What are the four stages of the Software Development Life Cycle? Name them in the correct order.

2. Briefly explain why testing and deployment often take more time than writing the code itself.
 3. Who was Grace Hopper, and what is her significance to the history of computer debugging?
 4. What was the cause of the Ariane 5 rocket explosion in 1996? What programming concept could have prevented it?
 5. Briefly describe the Knight Capital trading disaster of 2012. What type of testing failure contributed to it?
 6. Define the term "**bug**" in the context of computer programming. Where does the term originally come from?
 7. Define **testing** and **debugging** in your own words. How are they different from each other?
 8. List the six steps of the detective debugging process in the correct order.
 9. Why is approaching a bug "like a detective" a useful mindset for programmers?
 10. What was the Therac-25, and why is it considered the most important example of why testing is not optional?
-

Topic 5.2.1: Error Types — Syntax, Runtime, Logic

1. Define a **syntax error** and give one example of Python code that contains a syntax error.
2. Define a **runtime error** and give one example of a situation that causes one.
3. Define a **logic error** and explain why it is harder to detect than a syntax error or runtime error.
4. What is a **stack trace**? Describe how you read one correctly.
5. Why is the caret symbol (^) in a Python error message helpful to a programmer?
6. Name five common Python runtime exceptions and describe when each one is raised.
7. Differentiate between a `ValueError` and a `TypeError`. Give a specific example of each.
8. Differentiate between an `IndexError` and a `KeyError`. Give a specific example of each.
9. Explain the restaurant analogy for the three types of programming errors (Syntax, Runtime, Logic).
10. What tool is most effective for detecting each of the three error types? Match each error type to its best detection tool.
11. A program that calculates the average of a list of numbers runs without errors when given a list but crashes when given an empty list. What type of error is this? What exception does Python raise?
12. Predict the type of error and the specific exception name for this code:
`result = "hello" + 5`. Explain why.

13. A function uses `>` instead of `>=` in a conditional check. What type of error is this? Why does Python not catch it automatically?
-

Topic 5.2.2: Unit Testing — Assertions, unittest, pytest

1. Define **unit testing** in two to three sentences. Why is it compared to a car factory testing individual parts?
 2. What is an **assertion** in Python? What happens when an assertion is True? What happens when it is False?
 3. What is the `unittest` module? Is it built-in or external? What class must a test class inherit from?
 4. List four `unittest` assertion methods and describe what each one checks.
 5. What is `pytest`? How is it different from `unittest` in terms of how you write test functions?
 6. How do you install `pytest`? Write the exact command.
 7. What does the `-v` flag do when running `pytest`?
 8. What is a **test case**? What does it contain?
 9. Explain what an **edge case** is and why it is important to include edge cases in your test suite.
 10. Write a list of five questions you should ask yourself when analyzing edge cases for any function.
 11. Differentiate between a **normal case**, an **edge case**, and an **invalid input case** in testing. Give an example of each for a function that calculates a percentage.
 12. What does it mean when `unittest` output shows `FAILED (failures=1)`? Is this always bad? Explain.
 13. In `pytest`, how do you write a test that verifies a function raises a `ZeroDivisionError` when dividing by zero?
 14. What is **regression testing** and why is it important after making changes to a program?
 15. Explain why a failing unit test is described as a "gift" rather than a punishment.
 16. What is a **test suite**? How is it different from a single test case?
 17. Why should you write test cases BEFORE looking at the implementation of a function (as in the "Grab a Partner" activity)?
-

Topic 5.2.3: Breakpoints and Watch Expressions

1. Define a **breakpoint** in your own words. What happens when program execution reaches a breakpoint?
 2. How do you set a breakpoint in VS Code? How do you set one in PyCharm?
 3. Define a **watch expression**. When are watch expressions updated during a debugging session?
 4. Describe the difference between **Step Over** and **Step Into** in a debugger.
 5. Describe the difference between **Step Out** and **Resume/Continue** in a debugger.
 6. What are two specific situations where using a debugger is more efficient than using `print()` statements?
 7. What is an **off-by-one error**? Give a short example of code that contains one.
 8. When paused at a breakpoint, what information can you see in the Variables panel of the debugger?
 9. Write the keyboard shortcuts for Step Over, Step Into, Step Out, and Resume in most IDEs.
 10. Why is it said that breakpoints give you "total visibility into program execution" compared to `print()` debugging?
-

Topic 5.2.4: Exception Handling

1. Define **exception handling** in two to three sentences.
2. Describe the purpose of each block in a `try-except-else-finally` structure: `try`, `except`, `else`, `finally`.
3. When does the `else` block execute? When does it NOT execute?
4. When does the `finally` block execute? Give one practical example of what code belongs in a `finally` block.
5. What is the `raise` keyword used for in Python? Give a short example.
6. Define a **custom exception**. What class must a custom exception inherit from?
7. Write the Python syntax to define a custom exception called `InvalidScoreError`.
8. How do you catch multiple different exception types in a single `try` block? Write the syntax.
9. What is the difference between `except Exception` and `except ValueError`?
10. Explain what happens if a `try` block raises an exception but no matching `except` block exists.

11. Why is the `finally` block especially important in database applications?
 12. How does exception handling relate to the Ariane 5 disaster? What specific block would have prevented the crash?
 13. Differentiate between **raising** an exception and **handling** an exception.
 14. In the context of exception handling, what does "graceful failure" mean?
 15. Can a `try` block have multiple `except` blocks? Explain with a brief example.
-

Topic 5.3: Profiling and Optimization

1. Define **profiling** in the context of Python programming.
 2. Define **performance bottleneck**. Why should you always fix the biggest bottleneck first?
 3. What is the `cProfile` module? What kind of information does it report?
 4. What is the `timeit` module? How is it different from `cProfile`?
 5. Explain the meaning of the `tottime` column in `cProfile` output.
 6. Explain the meaning of the `cumtime` column in `cProfile` output.
 7. Define **refactoring**. What must remain unchanged after you refactor code?
 8. What is a **memory leak**? Why is it dangerous in a long-running program?
 9. Explain the difference between a Python **list** and a Python **generator** in terms of memory usage.
 10. Why should you NEVER optimize code before profiling it first?
 11. Why is `if item in my_set` faster than `if item in my_list` for large data collections?
 12. What is the time complexity of a nested loop where both inner and outer loops run `n` times? What does this mean for performance?
 13. Explain why Python's built-in functions like `sum()`, `max()`, and `min()` are faster than equivalent Python loops.
 14. After refactoring code to make it faster, what MUST you always do before finishing?
 15. Describe the F1 pit stop analogy for profiling. What does each part of the analogy represent in programming?
-

Section C: Long / Extensive Questions (LQs)

Topic 5.1: Introduction — Software Life Cycle and Debugging Mindset

1. The Ariane 5 rocket explosion in 1996 and the Therac-25 radiation machine failures are two of the most famous software disasters in history.
 - a) Describe the technical cause of the Ariane 5 explosion. Which specific programming concept, if applied, could have directly prevented the disaster?
 - b) Describe the cause of the Therac-25 failures. What type of testing failure contributed to the tragedy?
 - c) Compare the two disasters: what do they have in common in terms of software development failures? What lessons can modern developers learn from each?
 - d) Discuss why software testing is an **ethical responsibility** in safety-critical systems such as medical devices, aviation, and self-driving cars — not just a technical best practice.
 1. The Software Development Life Cycle consists of four stages.
 - a) Name and describe each stage in the correct order.
 - b) Explain why testing and deployment often take more time than writing the code itself. Support your answer with at least two reasons.
 - c) How do testing and debugging relate to each other? Are they the same process? Explain the key difference.
 - d) A beginner programmer says: *"I'll just write the code and fix bugs as users report them."* Evaluate this approach. What are the specific risks and costs of finding bugs in production versus finding them during testing?
 1. The "detective mindset" is described as the correct approach to debugging.
 - a) List and explain each step of the six-step detective debugging process.
 - b) For each step, describe a real tool or technique from this chapter that supports that step (e.g., which tool supports the "TEST" step?).
 - c) Explain why randomly changing code and hoping it fixes the bug is an inferior approach to the detective process. What specific problems does random guessing create?
-

Topic 5.2.1: Error Types — Syntax, Runtime, Logic

1. Python programs can contain three types of errors: syntax errors, runtime errors, and logic errors.
 - a) Define each error type precisely and explain when Python detects it.
 - b) For each error type, provide a complete Python code example (at least 5 lines) that demonstrates the error.

c) Explain which tool is most effective for finding each type of error and why.

d) A student says: *"Logic errors are not as serious as runtime errors because the program doesn't crash."* Evaluate this statement. Do you agree or disagree? Justify your answer with reference to real-world consequences.

1. Stack traces are essential tools for understanding runtime errors.

a) Define a stack trace. Why should you always read a stack trace from the **bottom up**?

b) A student encounters this stack trace:

```
Traceback (most recent call last):
```

```
  File "school.py", line 15, in <module>
```

```
    result = process_grades(students)
```

```
  File "school.py", line 8, in process_grades
```

```
    average = total / count
```

```
ZeroDivisionError: division by zero
```

Walk through this stack trace line by line. Identify: (i) which file contains the bug, (ii) which function contains the bug, (iii) which line the error occurs on, and (iv) what the bug is.

c) Explain how you would fix the bug identified in part (b) using both: (i) a conditional check before the division, and (ii) a `try-except` block.

d) Create a table listing at least six common Python exceptions, when each is raised, and one example code snippet for each.

1. An off-by-one error is one of the most common logic errors in programming.

a) Define an off-by-one error precisely. Why is it classified as a logic error rather than a syntax or runtime error?

b) Write a Python function `count_items(data)` that is supposed to return the number of items in a list. Write a version with an off-by-one error, and then write the corrected version.

c) Describe the exact step-by-step debugger process you would use to find the off-by-one error in part (b), including: (i) where you place the breakpoint, (ii) which debugger action you use (Step Over or Step Into), and (iii) what variable values you check in the Watch panel.

d) Write a `pytest` test suite with at least five test cases that would catch the off-by-one error in part (b) before you even run the debugger.

Topic 5.2.2: Unit Testing — Assertions, unittest, pytest

1. Unit testing is a fundamental practice in professional software development.

a) Define unit testing. Explain the car factory analogy in detail — what do the "parts," the "workbench," and the "final assembly" represent in software development terms?

b) Explain the role of the `assert` keyword. What does it do when the condition is True? When False? Why is it the foundation of automated testing?

c) Write a complete Python function `calculate_bmi(weight_kg, height_m)` that returns BMI rounded to 2 decimal places and raises `ValueError` for non-positive weight or height.

d) Write a complete `unittest` test class for `calculate_bmi()` with at least eight test cases covering: normal values, boundary values (minimum valid input), and invalid inputs. Include the `if __name__ == "__main__": unittest.main()` block.

e) Explain what each `unittest` method you used does (e.g., `assertEqual`, `assertRaises`, `assertAlmostEqual`).

1. The `pytest` framework simplifies writing test cases compared to `unittest`.

a) Compare `unittest` and `pytest` on the following dimensions: (i) how tests are organized, (ii) how assertions are written, (iii) how exceptions are tested, (iv) how tests are discovered and run.

b) Write the complete `pytest` test file for the `calculate_discount(price, discount_percent)` function from the chapter. Include: two normal cases, two boundary edge cases (0% and 100% discount), two invalid input cases (negative price, discount > 100), and one decimal result test.

c) Explain what output you would see if the `calculate_discount()` function had a bug where it returned `price` unchanged instead of applying the discount. Show the exact failing test output format.

d) A student says: "I tested my function with input (100, 20) and it returned 80.0, so my function is correct." Is this sufficient testing? Explain what additional test cases are needed and why.

1. Edge case analysis is a critical skill in software testing.

a) Define an **edge case** and explain why normal test cases alone are insufficient for verifying software correctness.

b) For a function `is_valid_password(password)` that accepts passwords between 8–20 characters containing at least one digit, describe all edge cases you would test. List at least eight test cases.

c) Write the complete `pytest` test file for `is_valid_password()`, including the function implementation and all test cases from part (b).

d) Explain the concept of **boundary value analysis** — why do bugs most frequently appear at the exact boundaries of valid input ranges rather than in the middle?

e) A student implements `is_valid_password()` but writes `len(password) > 8` instead of `len(password) >= 8`. Which specific test case from part (b) would catch this bug? Explain how.

Topic 5.2.3: Breakpoints and Watch Expressions

1. IDE debuggers with breakpoints and watch expressions are powerful tools for finding bugs.

a) Define a **breakpoint** and explain precisely what happens when program execution reaches one.

b) Describe step-by-step how to set a breakpoint and start a debugging session in **both** VS Code and PyCharm.

c) Explain the purpose and practical use of **watch expressions**. How do you add a watch expression in VS Code? Give a specific example of a useful watch expression for debugging a loop.

d) Create a complete step-by-step debugger trace table for this buggy function:

```
python
def sum_positive(numbers):
    total = 0
    for i in range(1, len(numbers)): # Bug here
        if numbers[i] > 0:
            total += numbers[i]
    return total
```

Use input `numbers = [-1, 3, -2, 5, 2]`. Show: line executing, debugger action, variable state, and notes for each step. Identify the root cause and provide the fix.

e) Explain why fixing the bug found in part (d) with a `pytest` test is better than relying on the debugger alone for future protection.

2. The four debugger navigation actions allow precise control over program execution.

a) Describe each of the four debugger actions — Step Over, Step Into, Step Out, and Resume — with a clear definition, keyboard shortcut, and a specific scenario where you would choose that action.

b) A student is debugging this code and pauses at line 3 with a breakpoint:

```
python
1: def get_letter_grade(score):
2:     if score >= 90: return "A"
3:     if score >= 80: return "B"
4:     if score >= 70: return "C"
5:     return "F"
6:
7: def process_student(name, score):
8:     grade = get_letter_grade(score)
```

```

9:     return f"{name}: {grade}"
10:
11: result = process_student("Aisha", 85)
12: print(result)

```

The student is paused at line 11. Describe what happens for each action: (i) Step Over on line 11, (ii) Step Into on line 11, (iii) Step Into on line 8 after entering `process_student`.

c) Explain the difference between debugging using `print()` statements and using a proper IDE debugger. Describe at least three advantages of the IDE debugger approach.

Topic 5.2.4: Exception Handling

1. Exception handling allows programs to respond to runtime errors gracefully.
 - a) Draw or describe the complete flow diagram of `try-except-else-finally` execution. Show the path when an exception occurs and the path when no exception occurs.
 - b) Write a complete Python program for a `StudentFileReader` class with a method `read_scores(filename)` that: reads a file of student scores (one per line), converts each to a float, and raises a meaningful error for each possible failure. Handle `FileNotFoundError`, `ValueError` (non-numeric data), and `PermissionError` each with a specific, helpful message. Use `finally` to ensure the file is closed.
 - c) Write a line-by-line execution trace table for your `read_scores()` method for the following two scenarios: (i) the file does not exist, and (ii) the file exists and contains valid data.
 - d) Explain why using specific exception types (`except ValueError`) is better practice than catching all exceptions with `except Exception`.
2. Custom exceptions make error handling more precise and readable.
 - a) Explain what a custom exception is, how you define one in Python, and why you would create a custom exception instead of using a built-in one.
 - b) Design a `BankAccount` class with `deposit()`, `withdraw()`, and `get_balance()` methods. Define three custom exceptions: `InsufficientFundsError`, `NegativeAmountError`, and `AccountFrozenError`. Write the complete class implementation with all custom exceptions properly raised.
 - c) Write a `pytest` test suite for `BankAccount` with at least eight test cases verifying: correct deposits, correct withdrawals, `InsufficientFundsError` for overdraft, `NegativeAmountError` for negative amounts, and frozen account behavior.
 - d) Explain the concept of **graceful failure** and how your `BankAccount` implementation demonstrates it. How does graceful failure improve the user experience compared to unhandled crashes?

3. Multiple exception types must be handled effectively in real-world programs.

a) Explain why a single function may need to handle multiple different exception types. Give a real-world example from a database, web API, or file processing context.

b) Write a complete function `parse_student_csv(filename, row_index)` that opens a CSV file, reads a specific row, extracts the student name and score, converts the score to a float, and calculates a letter grade. Handle: `FileNotFoundError`, `IndexError`, `ValueError`, and `ZeroDivisionError` — each with a specific, helpful message. Use `finally` to close the file.

c) Trace through your function for three different failure scenarios: (i) file not found, (ii) row index out of range, (iii) score is the string `"N/A"`. Show which exception is raised and which `except` block handles it in each case.

d) Evaluate the practice of using a single broad `except Exception` catch-all handler. What are its advantages and disadvantages compared to catching specific exception types?

Topic 5.3: Profiling and Optimization

1. Profiling is the essential first step before any performance optimization.

a) Define profiling. Explain the F1 race car pit stop analogy in detail — what does each element of the analogy represent in programming terms?

b) Explain the `cProfile` module. What does each column in its output report mean: `ncalls`, `tottime`, `percall`, `cumtime`?

c) Write a complete Python program that: (i) defines a `slow_search(data, target)` function using linear search, (ii) defines a `fast_search(data, target)` function using binary search, (iii) uses `timeit` to compare both on a list of 1,000,000 elements, and (iv) runs `cProfile` to generate a full profiling report.

d) Interpret the profiling output from part (c). Which function is the bottleneck? What is the approximate speed difference? Why is binary search faster — explain the "halving the search space" principle mathematically.

e) After identifying `slow_search` as the bottleneck, describe the complete Professional Optimization Cycle you would follow to fix it and verify the fix.

2. Refactoring strategies can dramatically improve program performance without changing program output.

a) Define **refactoring**. Why must the program output remain identical after refactoring? What is the role of unit tests in verifying this?

- b) Explain why nested loops create performance bottlenecks. If an outer loop runs `n` times and an inner loop runs `n` times, how many total operations occur? What time complexity is this called?
- c) Write a complete Python program that: (i) implements `find_common_elements_slow(list1, list2)` using nested loops, and (ii) implements `find_common_elements_fast(list1, list2)` using set intersection. Use `timeit` to compare both on lists of 10,000 elements. Show the complete code and expected output.
- d) Create a comparison table showing the time complexity of the following operations for lists vs. sets: membership check (`in`), adding an element, accessing by index, finding the minimum value.
- e) A student's program processes 1,000,000 customer records and takes 45 seconds. cProfile shows: `load_records()`: 42 seconds, `validate_records()`: 2.5 seconds, `format_output()`: 0.5 seconds. The student plans to optimize `format_output()` first because it is the easiest function to improve. Evaluate this decision. What should the student do instead, and why?
3. Memory profiling and generator expressions offer an alternative dimension of optimization.
- a) Define **memory profiling**. What is the `memory_profiler` library and what does it measure?
- b) Explain the difference between a Python list and a Python generator in terms of memory allocation. Why does `list(range(1_000_000))` use approximately 8MB of RAM while `range(1_000_000)` uses only about 48 bytes?
- c) Write a complete Python program that demonstrates the memory difference between processing a large dataset using: (i) a list comprehension, and (ii) a generator expression. Include comments explaining each approach.
- d) Describe three real-world scenarios where choosing a generator over a list would make a significant difference in program performance.
- e) Explain the concept of **peak memory usage** and why it matters for programs running on devices with limited RAM (e.g., mobile phones, embedded systems, IoT devices).
4. Comprehensive optimization case study — designing a high-performance student grade analyzer.
- a) A school has 50,000 student records stored as a list of dictionaries. Each record contains: `student_id`, `name`, `scores` (a list of 10 test scores). You must write a function `find_top_students(records, threshold)` that returns all students whose average score exceeds the `threshold`. Write two versions: a slow version using nested loops and list operations, and a fast version using optimized data structures and built-in functions.

- b) Write a complete **cProfile**-enabled program that runs both versions on 50,000 records and generates a comparative profiling report.
- c) Write a complete **pytest** test suite for both versions of **find_top_students()** that verifies: (i) both return identical results, (ii) edge cases (empty list, all students below threshold, all above threshold, threshold = 0, threshold = 100), and (iii) invalid inputs.
- d) After optimizing the function, describe the complete workflow — from profiling to refactoring to testing — that guarantees both correctness and performance. Explain why the testing step must come AFTER the optimization step.
- e) Evaluate the trade-off between **code readability** and **performance** in optimization. Is the fastest code always the best code? When might a slightly slower but more readable solution be preferable in a professional team environment?

End of Chapter 5 — Comprehensive Question Bank

Assessment Coverage Summary

Section	MCQs	Short Questions	Long Questions
5.1 Introduction & Life Cycle	15	10	3
5.2.1 Error Types	28	13	3
5.2.2 Unit Testing	26	17	3
5.2.3 Breakpoints & Watches	16	10	2
5.2.4 Exception Handling	18	15	3
5.3 Profiling & Optimization	22	15	4
TOTAL	120 MCQs	80 SQs	18 LQs

Total Questions: 218

Bloom's Taxonomy Coverage: Knowledge · Comprehension · Application · Analysis · Synthesis · Evaluation